SiMS-DeI / SICL — Manual operativo desde cero

Estado: documento operativo independiente

Sistema: SiMS-DeI — Sistema de Inteligencia de Diseño Espacial Multiescala

Lenguaje: SICL — Spatial Intelligence Command Language

Baseline: `sicl-core main@13b75f179985db6a00614d12f76de10f269b2be9`

Propósito. Este documento explica cómo operar el Core mediante tareas concretas. Describe comandos CLI, endpoints HTTP, salidas esperadas y errores frecuentes. No concede autoridad al sistema: una recomendación sigue siendo distinta de una decisión, y una decisión requiere revisión humana, actor y autoridad.

Cómo leer este manual

Cada tarea contiene cinco elementos: cuándo usarla, el comando SICL, el endpoint equivalente cuando existe, un resultado esperado y los errores que conviene revisar. Los ejemplos usan el proyecto `UPAO-001` y el actor `architect-yvan`.

Los comandos se ejecutan con un proyecto abierto. El proyecto activo es el contexto que determina dónde se guarda una mutación. Cuando un endpoint recibe `project_id`, el adaptador HTTP abre ese proyecto internamente y devuelve un envelope HTTP v1.

Los ejemplos de salida son ilustrativos. Los identificadores, timestamps, hashes y versiones pueden cambiar en cada ejecución.

Parte 1 — Onboarding

1.1 Qué necesitas antes de empezar

Necesitas Python 3.11 o una versión compatible, SQLite y las dependencias de `requirements.txt`. Para usar HTTP necesitas FastAPI, Uvicorn y un cliente que pueda enviar JSON. Para operaciones protegidas en producción también necesitas conocer el token de servicio configurado en `SICL_CORE_SERVICE_TOKEN`.

El Core distingue tres superficies. La CLI opera directamente sobre el repositorio SQLite. La API tradicional expone rutas de compatibilidad. La superficie canónica usa `/v1/...`, envelopes versionados y validación Pydantic.

1.2 Arrancar el Core con Python

Instala las dependencias:

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
```

Usa una ruta explícita para la base si necesitas separar una sesión de pruebas:

```
export SICL_DB_PATH=./sicl.sqlite
export SICL_CORS_ORIGINS=http://localhost:3000
PYTHONPATH=src uvicorn api.main:app --host 127.0.0.1 --port 8000
```

La aplicación FastAPI incorpora CORS con los orígenes de `SICL_CORS_ORIGINS`. No pongas el token de servicio en un bundle de navegador.

1.3 Arrancar con Docker

Cuando el repositorio tenga una imagen construida para el Core, el patrón operativo es:

```
docker build -t sicl-core .
docker run --rm -p 8000:8000 \
  -e SICL_DB_PATH=/data/sicl.sqlite \
  -e SICL_CORS_ORIGINS=http://localhost:3000 \
  -v "$PWD/data:/data" \
  sicl-core
```

El volumen es importante cuando el servicio se reinicia. SQLite conserva estado en el archivo indicado por `SICL_DB_PATH`; una instancia efímera puede perder datos aunque el proceso arranque correctamente.

1.4 Verificar que funciona

Comprueba el endpoint de salud no versionado:

```
curl -s http://127.0.0.1:8000/health
```

Salida esperada:

```
{"status":"ok","service":"sicl-core-api"}
```

Si existe token configurado, verifica la superficie canónica con el header correspondiente:

```
curl -s http://127.0.0.1:8000/v1/health \
  -H 'Authorization: Bearer '$SICL_CORE_SERVICE_TOKEN
```

1.5 Primer comando

Abre una sesión Python y crea un intérprete sobre un repositorio SQLite, o usa el entrypoint CLI del proyecto. El primer comando seguro es:

```
/HELP
```

La salida debe mostrar los comandos disponibles. Si no hay proyecto activo, el siguiente paso es crear uno.

1.6 Primer error resuelto

El error más común al comenzar es ejecutar una mutación sin proyecto abierto. Resuélvelo así:

```
/PROJECT CREATE UPAO-001 "Proyecto UPAO"
/PROJECT OPEN UPAO-001
```

Si el identificador ya existe, no repitas la creación. Usa `/PROJECT OPEN UPAO-001` y continúa desde el snapshot existente.

Parte 2 — Tareas frecuentes

T-01 — Crear proyecto

Cuándo usarla. Al iniciar un expediente espacial nuevo.

CLI.

```
/PROJECT CREATE UPAO-001 "Campus UPAO" parcela_sitio proyecto
```

HTTP equivalente. `POST /v1/projects`.

```
{
  "project_id":"UPAO-001",
  "name":"Campus UPAO",
  "spatial_scope":{"scale":"parcela_sitio","id":"UPAO-SITE","name":"Campus UPA
  "temporal_scope":{"horizon":"proyecto"},
  "actor":"architect-yvan"
}
```

Salida esperada. `code=OK`, `status=OK`, `project_id=UPAO-001` y `observed_version=1`.

Errores. `PROJECT_ALREADY_EXISTS`, payload vacío o escala no reconocida.

T-02 — Declarar escala

Cuándo usarla. Cuando el proyecto cambia de escala espacial o necesita un horizonte temporal explícito.

CLI.

```
/PROJECT SET SCOPE ciudad_distrito mediano_plazo
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands` con `command` igual al comando anterior.

Salida esperada. Un evento `SCOPE_CHANGED` y una versión incrementada.

Errores. `PROJECT_NOT_FOUND`, `INVALID_SCOPE` o `INVALID_STATE` si el proyecto está cerrado.

T-03 — Registrar hecho

Cuándo usarla. Para registrar una afirmación observada con una fuente identificable.

CLI.

```
/FACT SET "El área del sitio es 2000 m2" CATASTRO
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands`.

Salida esperada. Un registro en `facts` y un evento de mutación.

Errores. Proyecto inexistente, proyecto cerrado o statement vacío. Un hecho no se convierte automáticamente en supuesto.

T-04 — Registrar supuesto

Cuándo usarla. Para declarar una premisa de trabajo que todavía no es un hecho verificado.

CLI.

```
/ASSUMPTION SET "La ocupación inicial será 80%" ESTIMACION
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands`.

Salida esperada. Un registro en `assumptions`, separado de `facts`.

Errores. `INVALID_INPUTS`, proyecto cerrado o falta de basis. No uses `/FACT SET` para ocultar una estimación.

T-05 — Registrar evidencia

Cuándo usarla. Para guardar una declaración capturada, su tipo y su procedencia antes de derivar análisis.

CLI.

```
/EVIDENCE ADD EVD-001 "El plano registra 2000 m2" DOCUMENT CATASTRO https://ex
```

HTTP equivalente. `POST /v1/projects/{project_id}/evidence`.

```
{
  "evidence_id":"EVD-001",
  "statement":"El plano registra 2000 m2",
  "evidence_type":"DOCUMENT",
  "source_id":"CATASTRO",
  "evidence_url":"https://example.org/plano",
  "state":"OBSERVED"
}
```

Salida esperada. Envelope v1 con `data.evidence` y hash de la declaración cuando no se suministra uno.

Errores. Tipo o estado no canónico, fuente inexistente o proyecto cerrado.

T-06 — Registrar fuente

Cuándo usarla. Antes de enlazar evidencia o datos externos a una procedencia reutilizable.

CLI.

```
/SOURCE ADD CATASTRO OFFICIAL "Catastro municipal" https://example.org/catastr
```

HTTP equivalente. `POST /v1/projects/{project_id}/sources`.

Salida esperada. `data.source` con `source_id`, `source_type`, `title` y URL opcional.

Errores. `SOURCE_ALREADY_EXISTS`, `INVALID_SOURCE_TYPE` y `SOURCE_NOT_FOUND` al consultar un identificador inexistente.

T-07 — Definir objetivo

Cuándo usarla. Para declarar qué se quiere maximizar o minimizar.

CLI.

```
/OBJECTIVE SET CLIMATE MAXIMIZE 80
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands`.

Salida esperada. `objectives.CLIMATE` con dirección explícita.

Errores. `OBJECTIVE_DIRECTION_REQUIRED` si se omite la dirección, o `INVALID_INPUTS` para una dirección distinta de `MAXIMIZE` y `MINIMIZE`.

T-08 — Definir restricción

Cuándo usarla. Para una condición binaria obligatoria del proyecto.

CLI.

```
/CONSTRAINT SET HEIGHT ⩽ 6 FLOORS
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands`.

Salida esperada. Restricción con `hard=true`.

Errores. El MVP no admite restricciones soft. Un proyecto cerrado también rechaza la operación.

T-09 — Añadir rol

Cuándo usarla. Para dejar explícito quién participa y con qué rol nominal.

CLI.

```
/ROLE ADD ARCHITECT "Yvan Castillo"
```

HTTP equivalente. `POST /v1/projects/{project_id}/commands`.

Salida esperada. Registro en `roles` y evento asociado.

Errores. Actor vacío, proyecto inexistente o proyecto cerrado.

T-10 — Crear alternativa

Cuándo usarla. Cuando ya existe un problema u objetivo y se necesita una opción de diseño explícita.

CLI.

```
/ALTERNATIVE CREATE CONVENCIONAL_A "Estrategia convencional" "Mampostería y co
```

HTTP equivalente. El endpoint de comandos o la ruta de generaciones cuando la alternativa procede de un generador.

Salida esperada. Alternative con parámetros declarados y estado inicial controlado.

Errores. Parámetros mal formados, identificador duplicado o proyecto cerrado.

T-11 — Evaluar alternativa

Cuándo usarla. Para registrar el valor de una alternativa frente a un objetivo.

CLI.

```
/EVALUATE CONVENCIONAL_A CLIMATE 72 PERCENT 0.85 USER_INPUT
```

HTTP equivalente. `POST /v1/projects/{project_id}/evaluations`.

```
{"alternative":"CONVENCIONAL_A","objective":"CLIMATE","value":72,"unit":"PERCE
```

Salida esperada. `data.evaluation` y una versión observada.

Errores. Alternative u Objective inexistentes, valor inválido o falta de fuente.

T-12 — Comparar alternativas

Cuándo usarla. Para describir una comparación entre dos o más alternativas sin decidir por el usuario.

CLI.

```
/COMPARE CONVENCIONAL_A BIOCLIMATIC_A
```

HTTP equivalente. `POST /v1/projects/{project_id}/comparisons`.

Salida esperada. `data.comparison` con alternativas y trade-offs declarados.

Errores. Menos de dos alternativas, alternativas inexistentes o datos insuficientes.

T-13 — Obtener recomendación

Cuándo usarla. Para obtener una síntesis analítica antes de la revisión humana.

CLI. Depende del flujo de recomendación activo; normalmente se consulta el consejo/council del proyecto:

```
/STATUS
```

HTTP equivalente. `GET /v1/projects/{project_id}/council`.

Salida esperada. Recomendación descriptiva, condiciones y limitaciones.

Errores. Datos insuficientes o conflicto entre resultados. La recomendación no crea una Decision.

T-14 — Registrar HumanReview

Cuándo usarla. Antes de registrar una decisión humana.

CLI.

```
/HUMAN REVIEW "Revisé la recomendación y sus condiciones" architect-yvan "Arch
```

HTTP equivalente. `POST /v1/projects/{project_id}/human-reviews`.

Salida esperada. `human_review.status=APPROVED` con actor y authority.

Errores. `HUMAN_AUTHORITY_REQUIRED`, actor ausente o estado no válido.

T-15 — Registrar Decision

Cuándo usarla. Solo después de una HumanReview aprobada del mismo actor y autoridad.

CLI.

```
/DECISION RECORD "Adoptar CONVENCIONAL_A sujeto a las condiciones revisadas" a
```

HTTP equivalente. `POST /v1/projects/{project_id}/decisions`.

Salida esperada. Decision persistida con `actor`, `authority` y vínculo a la revisión humana.

Errores. `HUMAN_REVIEW_REQUIRED`, `HUMAN_AUTHORITY_REQUIRED` o proyecto cerrado. El Core nunca decide automáticamente.

T-16 — Ver snapshot

Cuándo usarla. Para leer el estado combinado del proyecto.

CLI.

```
/PROJECT SHOW
```

HTTP equivalente. `GET /v1/projects/{project_id}/snapshot`.

Salida esperada. Project, entidades, versión y estados de conocimiento.

Errores. `PROJECT_NOT_FOUND` o token ausente en HTTP protegido.

T-17 — Ver historial

Cuándo usarla. Para reconstruir las mutaciones en orden temporal.

CLI.

```
/HISTORY
```

HTTP equivalente. `GET /v1/projects/{project_id}/history`.

Salida esperada. Lista append-only ordenada por inserción temporal.

Errores. Proyecto inexistente o base SQLite no disponible.

T-18 — Simulación determinista

Cuándo usarla. Para ejecutar una simulación reproducible con parámetros declarados.

CLI.

```
/SIMULATE RUN DETERMINISTIC deterministic_basic_v1 '{"alternative_id":"CONVENC
```

HTTP equivalente. `POST /v1/projects/{project_id}/simulations`.

Salida esperada. Simulation append-only con `state=EXECUTED` o `INSUFFICIENT` si faltan entradas.

Errores. `METHOD_NOT_FOUND`, `METHOD_TYPE_MISMATCH`, `INVALID_INPUTS`.

T-19 — Monte Carlo

Cuándo usarla. Para cuantificar dispersión bajo una distribución declarada y semilla reproducible.

CLI.

```
/SIMULATE MONTE_CARLO CONVENCIONAL_A CLIMATE AREA '{"distribution":"NORMAL","m
```

HTTP equivalente. `POST /v1/projects/{project_id}/simulations` con `simulation_type=MONTE_CARLO` y `method=monte_carlo_v1`.

Salida esperada. `mean`, `std_dev`, `min`, `max`, `p10`, `p50`, `p90`, `samples_count`, `seed` y `provenance`.

Errores. Distribución no admitida, parámetro inexistente, iteraciones fuera de `1..10000`.

T-20 — Pareto

Cuándo usarla. Para obtener un frente descriptivo entre objetivos múltiples.

CLI.

```
/MULTIOBJECTIVE PARETO ENERGY_SAVINGS CONSTRUCTION_COST
```

HTTP equivalente. `POST /v1/projects/{project_id}/multiobjective/pareto`.

Salida esperada. Resultado con frente Pareto, alternativas incompletas y estado.

Errores. `OBJECTIVE_DIRECTION_REQUIRED`, evaluaciones faltantes o menos de dos objetivos.

T-21 — Trade-offs

Cuándo usarla. Para describir compromisos entre dos objetivos.

CLI.

```
/MULTIOBJECTIVE TRADEOFFS ENERGY_SAVINGS CONSTRUCTION_COST
```

HTTP equivalente. `POST /v1/projects/{project_id}/multiobjective/tradeoffs`.

Salida esperada. Trade-offs explícitos con inputs hash y sin decisión automática.

Errores. Objetivos inexistentes, dirección ausente o datos insuficientes.

T-22 — Generación paramétrica

Cuándo usarla. Para proponer candidatos a partir de una cuadrícula de parámetros.

CLI.

```
/GENERATE DESIGN parametric_grid_v1 '{"parameters":{"FLOORS":[4,6],"ORIENTATIO
```

HTTP equivalente. `POST /v1/projects/{project_id}/generations`.

Salida esperada. GeneratedAlternative en estado de propuesta, no Alternative promovida.

Errores. `INSUFFICIENT_INPUTS`, `METHOD_NOT_FOUND` o JSON inválido.

T-23 — Generación evolutiva

Cuándo usarla. Para explorar candidatos mediante semilla, población y generaciones declaradas.

CLI.

```
/GENERATE DESIGN evolutionary_v1 '{"seed":42,"population_size":8,"generations"
```

HTTP equivalente. `POST /v1/projects/{project_id}/generations`.

Salida esperada. Candidatos reproducibles con `generation_hash`.

Errores. Límites inválidos, inputs críticos ausentes o método desconocido.

T-24 — Consultar principios

Cuándo usarla. Para leer el catálogo read-only de principios de diseño.

CLI.

```
/DESIGN PRINCIPLES
/DESIGN PRINCIPLE DAYLIGHT_001
```

HTTP equivalente. `GET /v1/design/principles` y `GET /v1/design/principles/{principle_id}`.

Salida esperada. Principios con fuente declarada.

Errores. `PRINCIPLE_NOT_FOUND` o filtro de categoría inválido.

T-25 — Registrar instrumento de planificación

Cuándo usarla. Para asociar un instrumento declarativo al proyecto sin convertirlo automáticamente en normativa.

CLI.

```
/PLANNING ADD PLAN-001 MASTER_PLAN "Plan urbano local" Trujillo https://exampl
```

HTTP equivalente. `POST /v1/projects/{project_id}/planning/instruments`.

Salida esperada. Instrumento con estado verificable y fuente.

Errores. Tipo desconocido, instrumento sin fuente o vínculo duplicado.

T-26 — Registrar regulación

Cuándo usarla. Para registrar una referencia normativa con jurisdicción y fuente.

CLI.

```
/REGULATION ADD REG-001 CODE-001 "Reglamento urbano" Trujillo https://example.
```

HTTP equivalente. `POST /v1/regulations`.

Salida esperada. Regulation inicialmente no verificada si no existe evidencia suficiente.

Errores. Código vacío, fuente ausente o estado incompatible.

T-27 — Crear snapshot normativo

Cuándo usarla. Para congelar el conjunto normativo consultado en una fecha.

CLI.

```
/SNAPSHOT CREATE SNAP-001 UPAO-001 2026-09-16
/SNAPSHOT FREEZE SNAP-001 architect-yvan
```

HTTP equivalente. `POST /v1/projects/{project_id}/normative-snapshots` y `POST /v1/normative-snapshots/{snapshot_id}/freeze`.

Salida esperada. Snapshot `FROZEN` e inmutable.

Errores. Regulation sin fuente, reviewer ausente o snapshot inexistente.

T-28 — Crear relación multiescala

Cuándo usarla. Para declarar explícitamente una relación entre proyecto padre e hijo.

CLI.

```
/SCALE RELATE CITY-001 UPAO-001 CONTAINS "El campus pertenece al distrito"
```

HTTP equivalente. `POST /v1/projects/{project_id}/scale-relations`.

Salida esperada. ScaleRelation append-only con actor y timestamp.

Errores. `INVALID_SCOPE_RELATION`, relación inexistente o dirección ancestro-descendiente inválida.

T-29 — Importar objetivo

Cuándo usarla. Para importar un objetivo desde un proyecto relacionado mediante `CONTAINS`.

CLI.

```
/PROJECT IMPORT OBJECTIVE CITY-001 CLIMATE
```

HTTP equivalente. `POST /v1/projects/{project_id}/import-objective`.

Salida esperada. Objetivo local con `source_parent_objective_id`.

Errores. Falta la relación espacial, objetivo inexistente o autoridad de origen no transferible.

T-30 — Registrar actor y posición

Cuándo usarla. Para distinguir actores del proyecto y su postura frente a un sujeto.

CLI.

```
/ACTOR ADD MUNICIPALITY ROLE "Municipalidad" PUBLIC
/POSITION ADD MUNICIPALITY ALTERNATIVE CONVENCIONAL_A SUPPORTS "Cumple el plan
```

HTTP equivalente. `POST /v1/projects/{project_id}/actors` y `POST /v1/projects/{project_id}/positions`.

Salida esperada. Actor y ActorPosition consultables.

Errores. Actor duplicado, sujeto inexistente o reason vacío.

T-31 — Crear ciclo temporal

Cuándo usarla. Para fijar objetivos y supuestos en un horizonte.

CLI.

```
/CYCLE CREATE CYCLE-2030 mediano_plazo 2026-01-01 2030-12-31
```

HTTP equivalente. `POST /v1/projects/{project_id}/cycles`.

Salida esperada. TemporalCycle persistido con fechas y horizonte.

Errores. Fecha inválida, ciclo duplicado o horizonte desconocido.

T-32 — Crear escenario ramificado

Cuándo usarla. Para explorar una rama sin reescribir el ciclo base.

CLI.

```
/SCENARIO CREATE SCENARIO-A CYCLE-2030 "Alta densidad" '{"growth":"high"}'
/SCENARIO SELECT SCENARIO-A architect-yvan "Scenario authority"
```

HTTP equivalente. `POST /v1/projects/{project_id}/scenarios` y `POST /v1/scenarios/{branch_id}/select`.

Salida esperada. ScenarioBranch seleccionada explícitamente.

Errores. Parent cycle inexistente, authority ausente o escenario duplicado.

T-33 — Registrar evolución temporal

Cuándo usarla. Para documentar una transición entre dos ciclos.

CLI.

```
/EVOLUTION CREATE EVO-001 SCENARIO-A CYCLE-2030 CYCLE-2040
/EVOLUTION ADD EVO-001 '{"growth":"medium","trigger":"new census"}'
/EVOLUTION APPLY EVO-001 architect-yvan "Scenario authority"
```

HTTP equivalente. `POST /v1/projects/{project_id}/scenario-evolutions`, `POST /v1/scenario-evolutions/{evolution_id}/apply`.

Salida esperada. Evolución append-only aplicada con actor y authority.

Errores. Ciclos incompatibles, evolución inexistente o `HUMAN_AUTHORITY_REQUIRED`.

T-34 — Extraer memoria

Cuándo usarla. Para convertir conocimiento revisado del proyecto en una entrada de memoria institucional.

CLI.

```
/MEMORY EXTRACT UPAO-001 architect-yvan "Architect with project authority"
```

HTTP equivalente. `POST /v1/memory/extract`.

Salida esperada. InstitutionalMemory con provenance y estado.

Errores. Falta de autoridad, datos insuficientes o proyecto inexistente.

T-35 — Aplicar memoria

Cuándo usarla. Para aplicar una memoria autorizada a otro contexto sin ocultar su procedencia.

CLI.

```
/MEMORY APPLY MEM-001 architect-yvan
```

HTTP equivalente. `POST /v1/projects/{project_id}/memory/apply`.

Salida esperada. Aplicación registrada con vínculo a la memoria original.

Errores. `MEMORY_NOT_FOUND`, memoria revocada o actor no autorizado.

Parte 3 — Flujos completos

Flujo 1 — Proyecto mínimo

Objetivo. Crear un proyecto, registrar conocimiento inicial y dejar una restricción explícita.

```
/PROJECT CREATE UPAO-001 "Campus UPAO" parcela_sitio proyecto
/PROJECT OPEN UPAO-001
/FACT SET "El área del sitio es 2000 m2" CATASTRO
/ASSUMPTION SET "La ocupación inicial será 80%" ESTIMACION
/OBJECTIVE SET CLIMATE MAXIMIZE 80
/CONSTRAINT SET HEIGHT ⩽ 6 FLOORS
/ROLE ADD ARCHITECT "Yvan Castillo"
/PROJECT SHOW
/HISTORY
```

Salida esperada. Snapshot con las entidades separadas, versión creciente y eventos ordenados. La verificación mínima es comprobar que el hecho aparece en `facts`, el supuesto en `assumptions`, y el constraint tiene `hard=true`.

Flujo 2 — Análisis multiobjetivo

Objetivo. Comparar opciones y observar el frente Pareto sin crear una decisión.

```
/ALTERNATIVE CREATE CONVENCIONAL_A "Convencional" "Mampostería" '{"FLOORS":4,"
/ALTERNATIVE CREATE BIOCLIMATIC_A "Bioclimática" "Protección solar" '{"FLOORS"
/EVALUATE CONVENCIONAL_A CLIMATE 72 PERCENT 0.85 USER_INPUT
/EVALUATE BIOCLIMATIC_A CLIMATE 88 PERCENT 0.85 USER_INPUT
/MULTIOBJECTIVE PARETO CLIMATE CONSTRUCTION_COST
/MULTIOBJECTIVE TRADEOFFS CLIMATE CONSTRUCTION_COST
```

Salida esperada. Resultado descriptivo, con alternativas incompletas si falta alguna evaluación. La verificación consiste en comprobar que no aparece `Decision` como efecto colateral.

Flujo 3 — Gobernanza

Objetivo. Mostrar el orden constitucional recomendación, revisión humana y decisión.

```
/STATUS
/HUMAN REVIEW "Revisé la recomendación y sus condiciones" architect-yvan "Archi
/DECISION RECORD "Adoptar BIOCLIMATIC_A sujeto a las condiciones revisadas" ar
/HISTORY
```

Salida esperada. La decisión queda asociada al actor, authority y revisión. La verificación negativa es intentar la decisión sin HumanReview y confirmar `HUMAN_REVIEW_REQUIRED`.

Flujo 4 — Multiescala

Objetivo. Vincular una escala territorial y transferir un objetivo sin copiar autoridad.

```
/PROJECT CREATE CITY-001 "Distrito de referencia" ciudad_distrito mediano_plaz
/SCALE RELATE CITY-001 UPAO-001 CONTAINS "El campus pertenece al distrito"
/PROJECT OPEN UPAO-001
/PROJECT IMPORT OBJECTIVE CITY-001 CLIMATE
/SCALE RELATIONS UPAO-001
```

Salida esperada. El objetivo importado conserva `source_parent_objective_id`. La relación se consulta como append-only.

Flujo 5 — Simulación y Monte Carlo

Objetivo. Ejecutar una simulación reproducible y consultar su registro.

```
/PROJECT OPEN UPAO-001
/SIMULATE METHODS
/SIMULATE MONTE_CARLO CONVENCIONAL_A CLIMATE AREA '{"distribution":"NORMAL","m
/SIMULATE LIST
/SIMULATE SHOW SIM-001
```

Salida esperada. Statistics, seed, sample count y evidence hash. Repetir con los mismos inputs y semilla debe producir el mismo resultado. Cambiar la semilla produce una ejecución distinta. Ninguna ejecución crea Decision o Recommendation.

Parte 4 — Errores frecuentes

Errores de dominio

`HUMAN_REVIEW_REQUIRED` significa que se intentó registrar una decisión sin una revisión humana aprobada del mismo contexto. `HUMAN_AUTHORITY_REQUIRED` indica que faltan actor o authority explícitos.

`PROJECT_NOT_FOUND` significa que el identificador no existe. `PROJECT_ALREADY_EXISTS` significa que se intentó crear un identificador usado. `INVALID_STATE` aparece cuando una transición o mutación no es válida, especialmente sobre un proyecto `CLOSED`.

`INVALID_INPUTS` indica que faltan datos o tienen forma inválida. `METHOD_NOT_FOUND` indica que el nombre del método no está catalogado. `METHOD_TYPE_MISMATCH` indica que el método no corresponde al tipo de simulación o generación solicitado.

`LLM_NOT_CONFIGURED` es el resultado esperado para `llm_assisted_v1` cuando no existe un proveedor configurado y autorizado. No se debe interpretar como una ejecución parcial.

`OBJECTIVE_DIRECTION_REQUIRED` aparece cuando un cálculo multiobjetivo no recibe `MAXIMIZE` o `MINIMIZE` de forma explícita. `INSUFFICIENT` indica que el sistema no tiene evidencia o evaluaciones suficientes y no debe completar silenciosamente el dato.

Errores HTTP

Un `401` indica que la API exige un token y no se envió `Authorization: Bearer <token>`, o el token no coincide. Un `422` normalmente indica una validación Pydantic o una regla semántica del Core. Un `404` identifica un proyecto, entidad o método inexistente. Un `409` indica conflicto de estado, duplicación o una operación incompatible.

Un envelope v1 exitoso tiene esta forma:

```
{
  "contract_version":"1.0",
  "status":"OK",
  "code":"OK",
  "message":"ok",
  "project_id":"UPAO-001",
  "observed_version":4,
  "data":{}
}
```

Un error conserva `contract_version`, `code`, `message`, `project_id` y `data` dentro del detalle HTTP. El cliente debe mostrar el código y no sustituirlo por un mensaje genérico.

Parte 5 — Referencia rápida

Comandos de proyecto y conocimiento

Área	Comandos
Proyecto	`/PROJECT CREATE`, `/PROJECT OPEN`, `/PROJECT SHOW`, `/PROJECT LIST`, `/PROJECT SET SCOPE`, `/PROJECT IMPORT OBJECTIVE`
Estado	`/STAGE SET`, `/STATUS`, `/HISTORY`
Epistemología	`/FACT SET`, `/ASSUMPTION SET`, `/EVIDENCE ADD`, `/EVIDENCE LIST`, `/EVIDENCE SHOW`, `/SOURCE ADD`, `/SOURCE LIST`, `/SOURCE SHOW`
Diseño	`/OBJECTIVE SET`, `/CONSTRAINT SET`, `/ROLE ADD`, `/ALTERNATIVE CREATE`, `/ALTERNATIVE SET`, `/ALTERNATIVE LIST`, `/ALTERNATIVE PROMOTE`
Análisis	`/EVALUATE`, `/COMPARE`, `/MULTIOBJECTIVE PARETO`, `/MULTIOBJECTIVE TRADEOFFS`, `/SIMULATE`, `/GENERATE DESIGN`
Gobernanza	`/HUMAN REVIEW`, `/DECISION RECORD`, `/COUNCIL` por HTTP, `/DEBATE` cuando está disponible

Comandos temporales, institucionales y normativos

Área	Comandos
Actores	`/ACTOR ADD`, `/ACTOR LIST`, `/ACTOR SHOW`, `/POSITION ADD`, `/POSITION LIST`
Tiempo	`/CYCLE CREATE`, `/CYCLE LIST`, `/CYCLE SHOW`, `/SCENARIO CREATE`, `/SCENARIO LIST`, `/SCENARIO SELECT`
Evolución	`/EVOLUTION CREATE`, `/EVOLUTION ADD`, `/EVOLUTION APPLY`, `/EVOLUTION LIST`, `/EVOLUTION SHOW`
Memoria	`/MEMORY LIST`, `/MEMORY SHOW`, `/MEMORY EXTRACT`, `/MEMORY REVOKE`, `/MEMORY APPLY`, `/MEMORY TYPES`
Planificación	`/PLANNING ADD`, `/PLANNING LIST`, `/PLANNING SHOW`, `/PLANNING TYPES`
Normativa	`/REGULATION ADD`, `/REGULATION LIST`, `/REGULATION SHOW`, `/REGULATION STATUS`, `/INTERPRET ADD`, `/INTERPRET LIST`, `/INTERPRET REVIEW`, `/SNAPSHOT CREATE`, `/SNAPSHOT LIST`, `/SNAPSHOT FREEZE`
Escalas	`/SCALE PARENT`, `/SCALE CHILDREN`, `/SCALE RELATE`, `/SCALE RELATIONS`

Endpoints HTTP principales

Método	Endpoint	Uso
GET	`/health`	Salud del servicio
GET/POST	`/v1/projects`	Listar o crear proyectos
GET	`/v1/projects/{id}/snapshot`	Leer snapshot
GET	`/v1/projects/{id}/history`	Leer historial
POST	`/v1/projects/{id}/commands`	Ejecutar comando canónico
POST/GET	`/v1/projects/{id}/evidence`	Crear o listar evidencia
POST/GET	`/v1/projects/{id}/sources`	Crear o listar fuentes
GET	`/v1/projects/{id}/sources/{source_id}`	Leer fuente
POST/GET	`/v1/projects/{id}/evaluations`	Crear o listar evaluaciones
POST/GET	`/v1/projects/{id}/comparisons`	Crear o listar comparaciones
POST	`/v1/projects/{id}/human-reviews`	Registrar revisión humana
POST	`/v1/projects/{id}/decisions`	Registrar decisión autorizada
GET	`/v1/projects/{id}/council`	Leer council/recomendación
POST	`/v1/projects/{id}/simulations`	Ejecutar simulación
GET	`/v1/simulations/methods`	Listar métodos
POST	`/v1/projects/{id}/multiobjective/pareto`	Calcular Pareto
POST	`/v1/projects/{id}/multiobjective/tradeoffs`	Calcular trade-offs
POST/GET	`/v1/projects/{id}/generations`	Generar o listar candidatos

Método	Endpoint	Uso
GET	`/v1/design/principles`	Catálogo read-only
POST/GET	`/v1/projects/{id}/planning/instruments`	Instrumentos de planificación
POST/GET	`/v1/regulations`	Corpus normativo
POST/GET	`/v1/projects/{id}/normative-snapshots`	Snapshots normativos
POST/GET	`/v1/projects/{id}/scale-relations`	Relaciones multiescala
POST	`/v1/projects/{id}/import-objective`	Importar objetivo
POST/GET	`/v1/projects/{id}/cycles`	Ciclos temporales
POST/GET	`/v1/projects/{id}/scenarios`	Escenarios
POST/GET	`/v1/projects/{id}/scenario-evolutions`	Evoluciones

Estados

Tipo	Valores operativos
Stage	`DRAFT`, `ACTIVE`, `CLOSED`
Knowledge state	`UNKNOWN`, `CONFLICTING`, `INSUFFICIENT`, `OBSERVED`
Objective direction	`MAXIMIZE`, `MINIMIZE`
Simulation	`DETERMINISTIC`, `MONTE_CARLO`, `SCENARIO`, `SENSITIVITY`
Monte Carlo distribution	`NORMAL`, `UNIFORM`, `TRIANGULAR`
Normative snapshot	`FROZEN` cuando queda congelado

Escalas espaciales

El catálogo espacial tiene nueve valores: `pais`, `region`, `provincia_metropoli`, `ciudad_distrito`, `barrio_sector`, `parcela_sitio`, `edificio`, `espacio` y `objeto`. El Core no infiere una escala desde un nombre de lugar. La escala se declara mediante el comando o el payload.

Los horizontes temporales son `proyecto`, `corto_plazo`, `mediano_plazo`, `largo_plazo`, `escenario_2030`, `escenario_2040` y `escenario_2050`. El horizonte por defecto es

`proyecto` cuando no se declara otro.

Reglas de operación que no se deben olvidar

1. `Fact` y `Assumption` son entidades diferentes.
2. `Recommendation` no es `Decision`.
3. `HumanReview` debe preceder a `Decision`.
4. Actor y authority deben ser explícitos en las acciones de autoridad.
5. Las simulaciones, comparaciones, evaluaciones, generaciones y agentes no deciden.
6. Events, Evidence, Simulation, GeneratedAlternative y relaciones temporales son append-only según su contrato.
7. Un proyecto cerrado no acepta nuevas mutaciones.
8. La normativa siempre conserva fuente, estado y disclaimer profesional.
9. Los estados `UNKNOWN`, `CONFLICTING` e `INSUFFICIENT` no se reemplazan silenciosamente.
10. Un error debe detener la operación, no producir un falso resultado exitoso.

Referencias

Firma documental. Este manual fue redactado como una perspectiva operativa independiente sobre el código y los contratos del baseline indicado. No sustituye el Core Contract ni la aprobación del Product Owner.